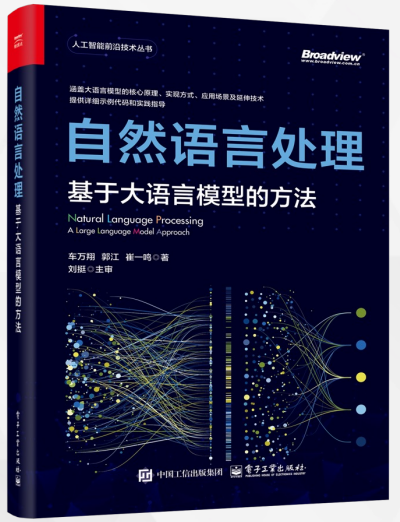


自然语言处理中的深度学习基础

车万翔

社会计算与交互机器人研究中心（HIT-SCIR）
哈尔滨工业大学

本讲内容与学习目标

课堂内容	学完后能够
输入表示与MLP	解释非线性，读懂张量形状
CNN与RNN	比较局部特征与序列记忆
注意力与Transformer	手算注意力，解释位置与Mask
训练与NLP实践	匹配损失函数，处理变长文本

影评情感分类

影评示例	标签	模型需要识别的信息
这部电影很好看	正向 1	情感词
这部电影并不好看	负向 0	否定与局部搭配
虽然开头慢，但结尾很精彩	正向 1	转折与上下文

共同任务：把一段文本序列，变成两个类别的得分。

感知器：加权评分与阈值判断

一个早期的线性二分类模型

加权评分 $z = w_1x_1 + w_2x_2 + b$

阈值判断 $z \geq 0$ 时预测1，否则预测0

示例 $0.6 \times 80 + 0.4 \times 60 - 70 = 2$

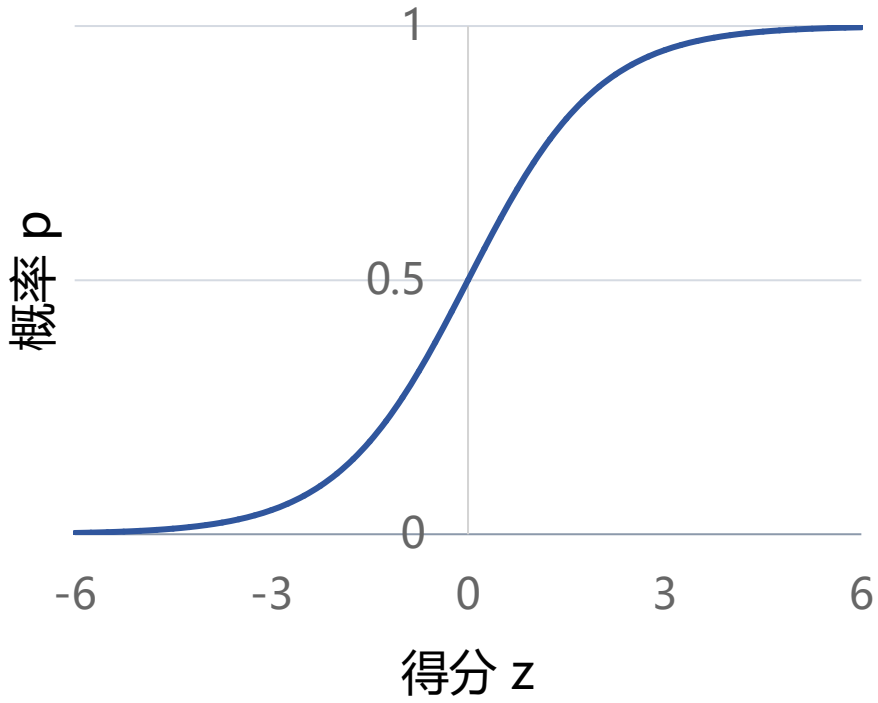


权重 w 决定各项特征的影响，偏置 b 调整判断门槛。

Logistic回归：用概率表达二分类

线性得分：$z = w^T x + b$
正类概率：$p = \sigma(z)$
$\sigma(z) = 1 / (1 + \exp(-z))$

z	−2	0	2
σ(z)	0.119	0.500	0.881

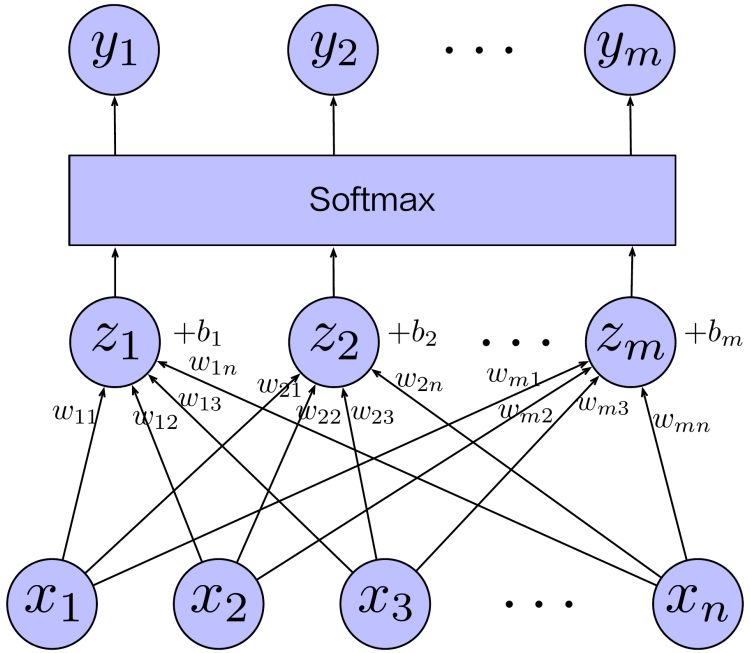


常用阈值为0.5，此时等价于判断 $z \geq 0$。

Softmax回归：多类别的概率分布

$$p_i = \exp(z_i) / \Sigma_j \exp(z_j)$$

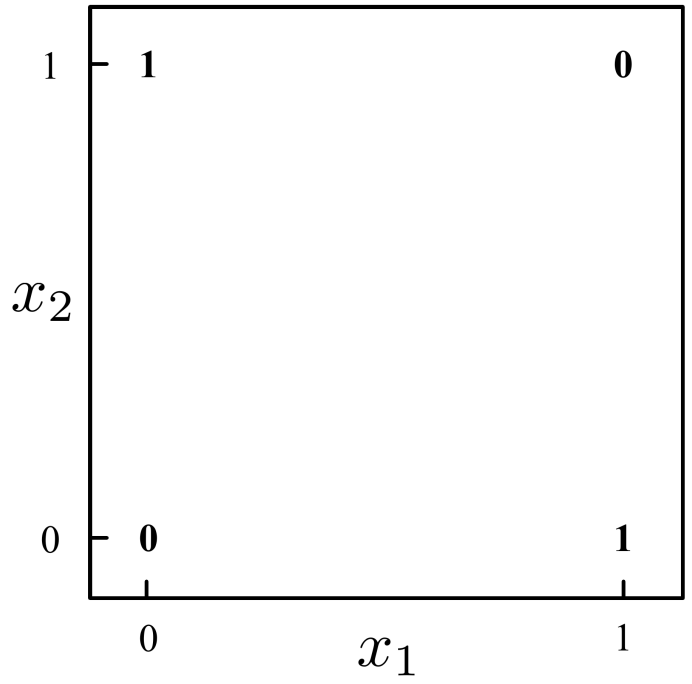
类别	正向	中性	负向
得分 z	2	1	0
概率 p	0.665	0.245	0.090



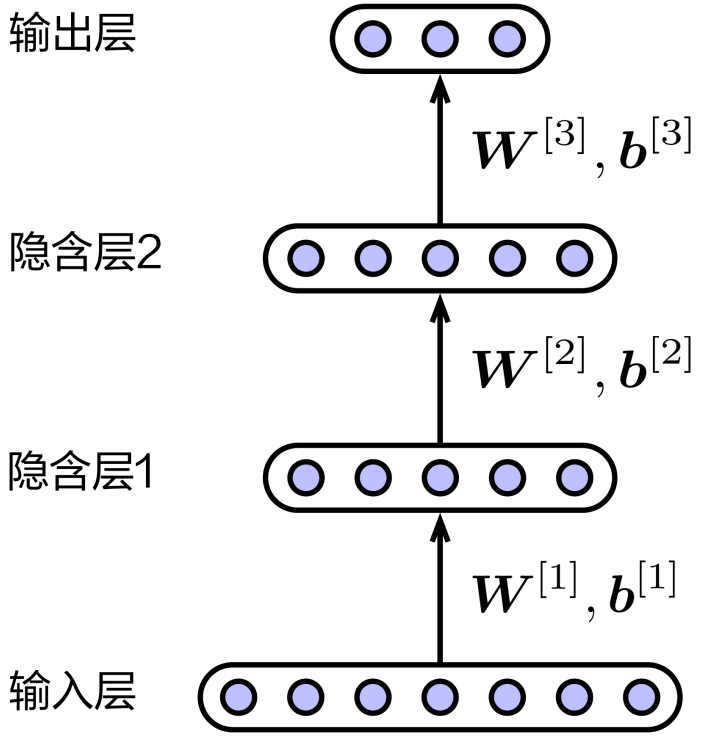
数值稳定：先令 z′ = z − max(z)
概率保持不变，避免指数溢出。

MLP：非线性让模型表达更复杂的关系

异或 XOR：输入不同时为1，相同时为0。

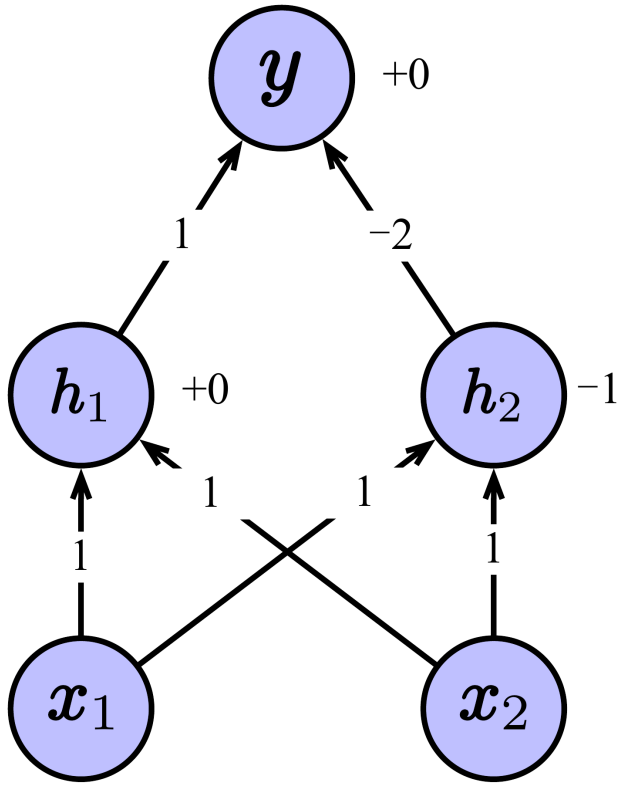


线性层
+ 非线性激活
+ 线性层



仅叠加线性层无法解决XOR。隐层需要非线性变换。

课堂手算：一个能实现XOR的小网络

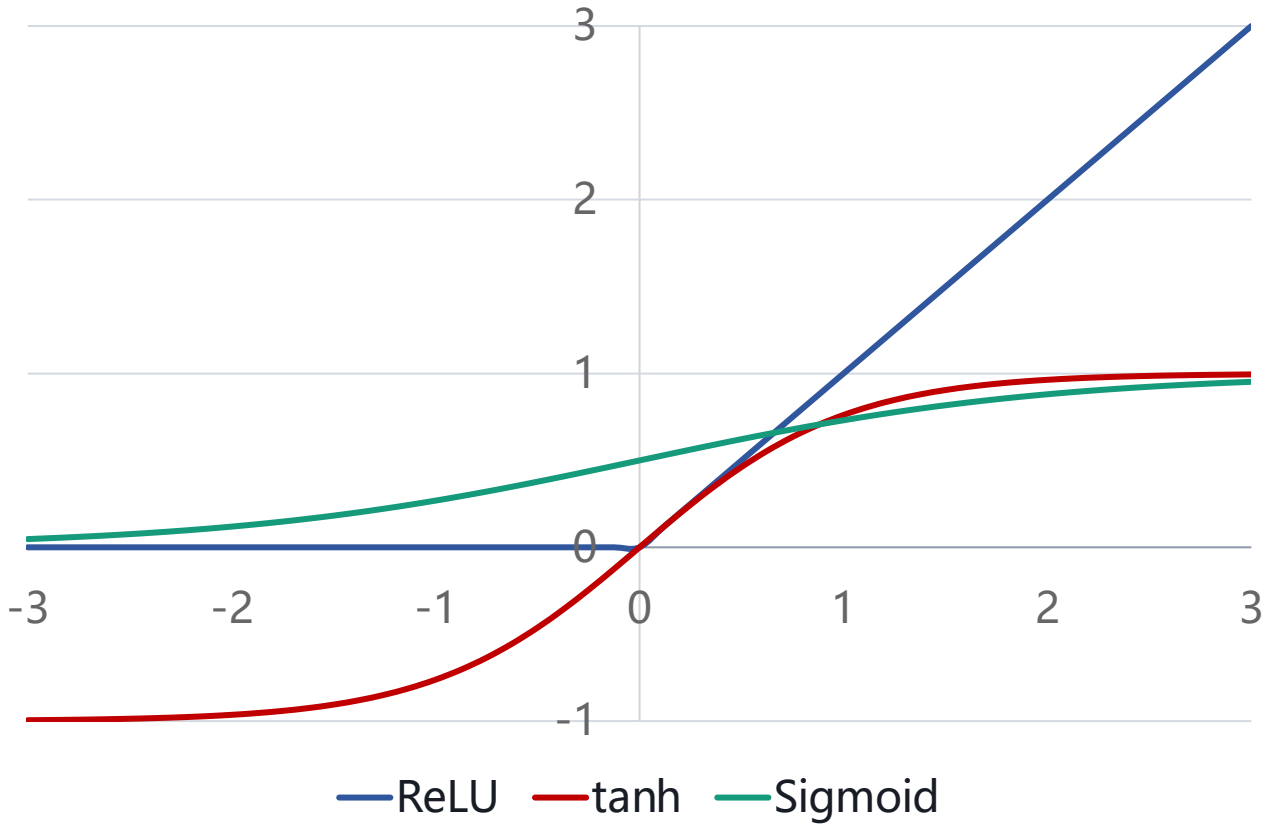


$h_1 = \mathrm{ReLU}(x_1 + x_2)$
$h_2 = \mathrm{ReLU}(x_1 + x_2 - 1)$
$y = h_1 - 2h_2$

(x_1, x_2)	(h_1, h_2)	y
(0, 0)	(0, 0)	0
(0, 1)	(1, 0)	1
(1, 0)	(1, 0)	1
(1, 1)	(2, 1)	0

ReLU(a) = max(0, a)

激活函数与非线性变换



ReLU 正值保留，负值归零

tanh 输出在 −1 与 1 之间

Sigmoid 输出在 0 与 1 之间

没有非线性时：$W_2(W_1x + b_1) + b_2$ 仍是一次仿射变换。

PyTorch实现：MLP的输入与输出

```
model = nn.Sequential(
    nn.Linear(8, 16),
    nn.ReLU(),
    nn.Linear(16, 2)
)
x = torch.randn(4, 8)
logits = model(x)
print(logits.shape)  # torch.Size([4, 2])
```

8维输入，16维隐层，2个类别得分。

训练使用 CrossEntropyLoss 时，模型输出层保留 logits

词向量与MLP：如何表示一个句子

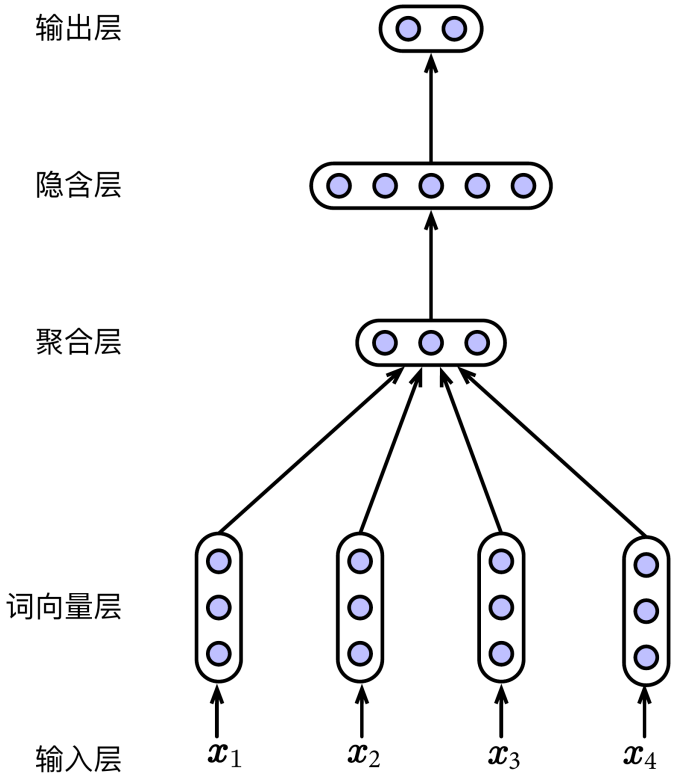
词向量　每个Token ID查找一个D维向量

平均池化　$h = (x_1 + \cdots + x_n) / n$

分类层　MLP(h) 输出两个类别得分

输入句子	词袋式平均表示
张三 喜欢 李四	相同
李四 喜欢 张三	相同

EmbeddingBag可直接聚合变长序列。平均池化时需要排除Padding。



卷积神经网络：寻找局部搭配

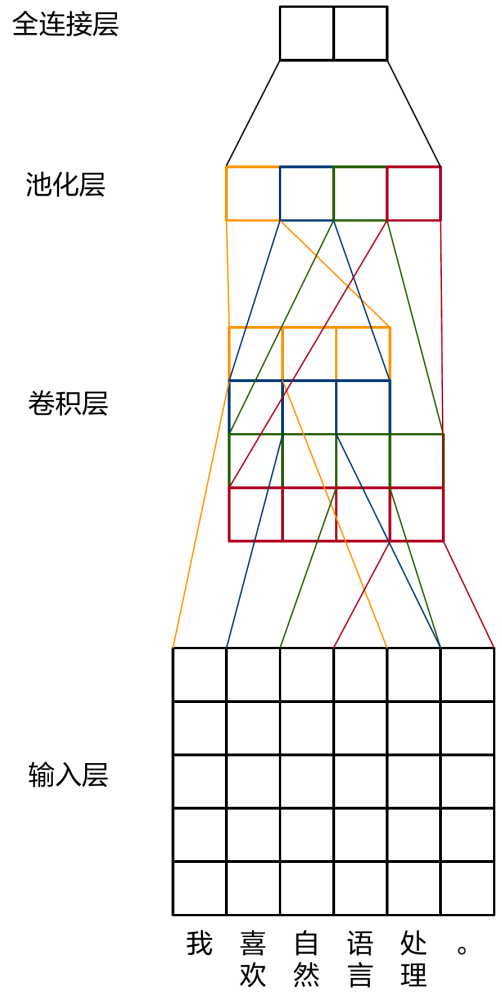
“不好看”比单独的“好看”提供更多信息

局部窗口　　按相邻词元的N-gram扫描文本

参数共享　　同一个卷积核用于不同位置

多个卷积核　　学习多种局部模式与不同窗口

池化　　将变长响应聚合为固定维向量



更深的卷积、扩张卷积或更大窗口可以扩大感受野。

一维卷积与最大池化：计算示例

输入 $x = [1, 2, 0, 3]$　卷积核 $w = [1, -1]$

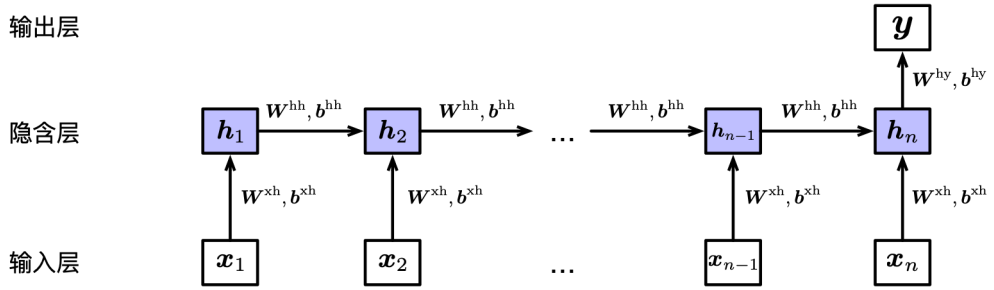
滑动窗口	加权计算	输出响应
[1, 2]	$1 \times 1 + 2 \times (-1)$	-1
[2, 0]	$2 \times 1 + 0 \times (-1)$	2
[0, 3]	$0 \times 1 + 3 \times (-1)$	-3

最大池化：$\max(-1, 2, -3) = 2$

本例步长1，无Padding和激活。输出长度为 $T - k + 1 = 3$。

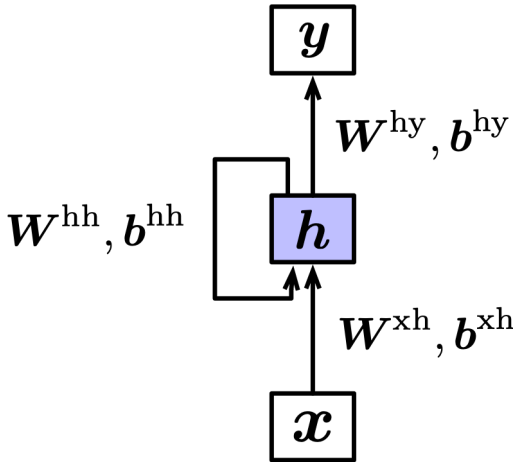
循环神经网络

- 卷积神经网络无法处理长距离的依赖
- 循环神经网络的解决方案
  - 每个时刻的隐状态依赖于当前时刻的输入以及上一时刻的隐状态

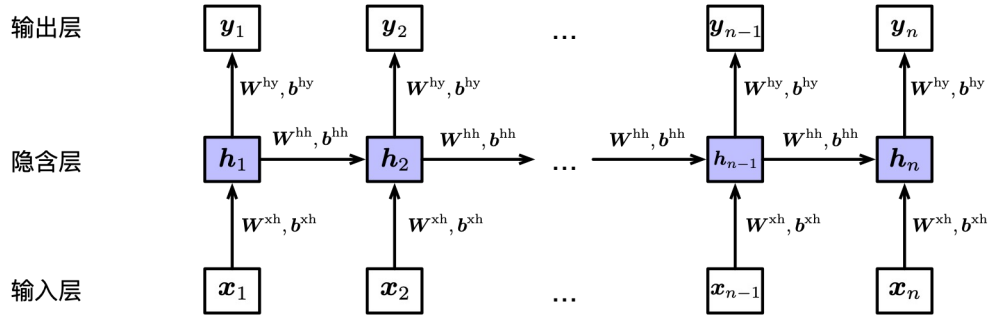


$$h_t = \tanh(W^{xh}x_t + b^{xh} + W^{hh}h_{t-1} + b^{hh})$$

$$y = \mathrm{Softmax}(W^{hy}h_n + b^{hy})$$



  - 每个时刻的参数共享（“循环”的由来）
  - 每个时刻也可以有相应的输出（序列标注）



colab

https://colab.research.google.com/drive/1mTeml7lAA9B2pS-O9yWm1_M7LiyHPPUJ?usp=sharing

长短时记忆网络（LSTM）

- 序列过长时，原始的循环神经网络容易导致信息损失
  - 梯度爆炸或梯度消散
- 新的加性隐状态更新方式

$$\boldsymbol{u}_t = \tanh(\boldsymbol{W}^{\mathrm{xh}}\boldsymbol{x}_t + \boldsymbol{b}^{\mathrm{xh}} + \boldsymbol{W}^{\mathrm{hh}}\boldsymbol{h}_{t-1} + \boldsymbol{b}^{\mathrm{hh}})$$

$$\boldsymbol{h}_t = \boldsymbol{h}_{t-1} + \boldsymbol{u}_t$$

long term　　short term

  - 相当于直接将 $\boldsymbol{h}_k$ 与 $\boldsymbol{h}_t$（$k < t$）进行了跨层连接
    - $\boldsymbol{h}_t = \boldsymbol{h}_{t-1} + \boldsymbol{u}_t = \boldsymbol{h}_{t-2} + \boldsymbol{u}_{t-1} + \boldsymbol{u}_t = \boldsymbol{h}_k + \boldsymbol{u}_{k+1} + \boldsymbol{u}_{k+2} + \cdots + \boldsymbol{u}_t$
- 进一步改进
  - 遗忘门：考虑旧状态 $\boldsymbol{h}_{t-1}$ 和新状态 $\boldsymbol{u}_t$ 的贡献

$$\boldsymbol{f}_t = \sigma(\boldsymbol{W}^{\mathrm{f,xh}}\boldsymbol{x}_t + \boldsymbol{b}^{\mathrm{f,xh}} + \boldsymbol{W}^{\mathrm{f,hh}}\boldsymbol{h}_{t-1} + \boldsymbol{b}^{\mathrm{f,hh}})$$

$$\boldsymbol{h}_t = \boldsymbol{f}_t \odot \boldsymbol{h}_{t-1} + (1 - \boldsymbol{f}_t) \odot \boldsymbol{u}_t$$

  - 输入门：独立控制 $\boldsymbol{h}_{t-1}$ 和 $\boldsymbol{u}_t$ 的贡献

$$\boldsymbol{i}_t = \sigma(\boldsymbol{W}^{\mathrm{i,xh}}\boldsymbol{x}_t + \boldsymbol{b}^{\mathrm{i,xh}} + \boldsymbol{W}^{\mathrm{i,hh}}\boldsymbol{h}_{t-1} + \boldsymbol{b}^{\mathrm{i,hh}})$$

$$\boldsymbol{h}_t = \boldsymbol{f}_t \odot \boldsymbol{h}_{t-1} + \boldsymbol{i}_t \odot \boldsymbol{u}_t$$

  - 输出门：对输出进行控制

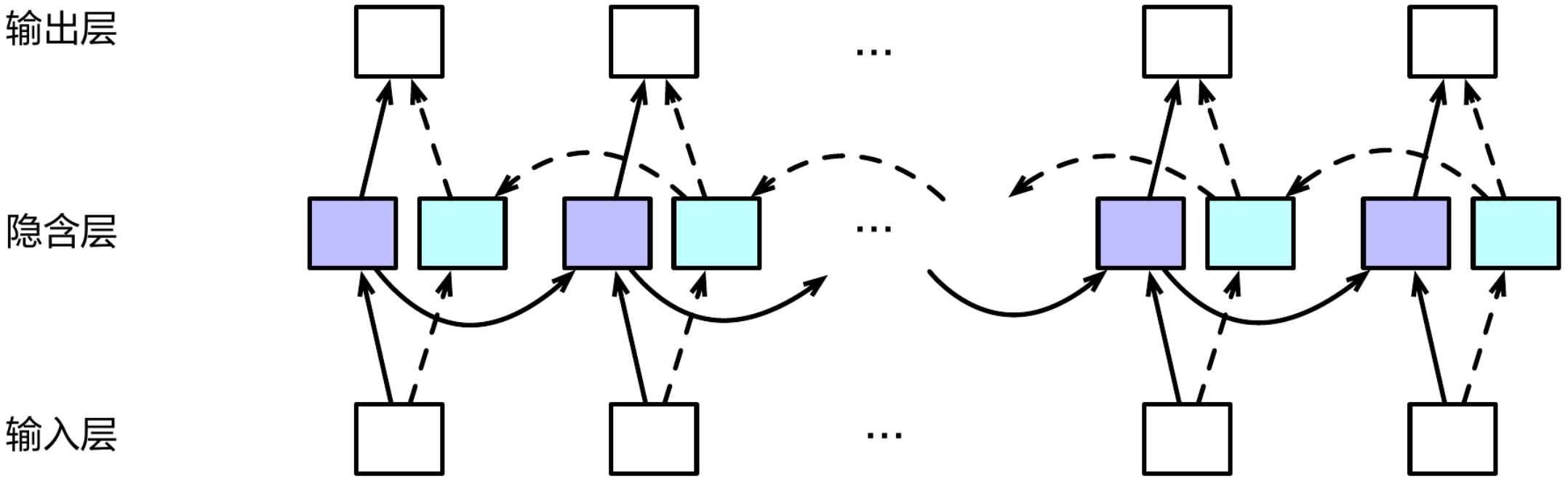
$$\boldsymbol{o}_t = \sigma(\boldsymbol{W}^{\mathrm{o,xh}}\boldsymbol{x}_t + \boldsymbol{b}^{\mathrm{o,xh}} + \boldsymbol{W}^{\mathrm{o,hh}}\boldsymbol{h}_{t-1} + \boldsymbol{b}^{\mathrm{o,hh}})$$

$$\boldsymbol{c}_t = \boldsymbol{f}_t \odot \boldsymbol{c}_{t-1} + \boldsymbol{i}_t \odot \boldsymbol{u}_t$$

$$\boldsymbol{h}_t = \boldsymbol{o}_t \odot \tanh(\boldsymbol{c}_t)$$

双向RNN与序列标注

整句已知时，一个词可以同时利用左右上下文



双向：同一层的两个方向。堆叠：前一层输出作为后一层输入。

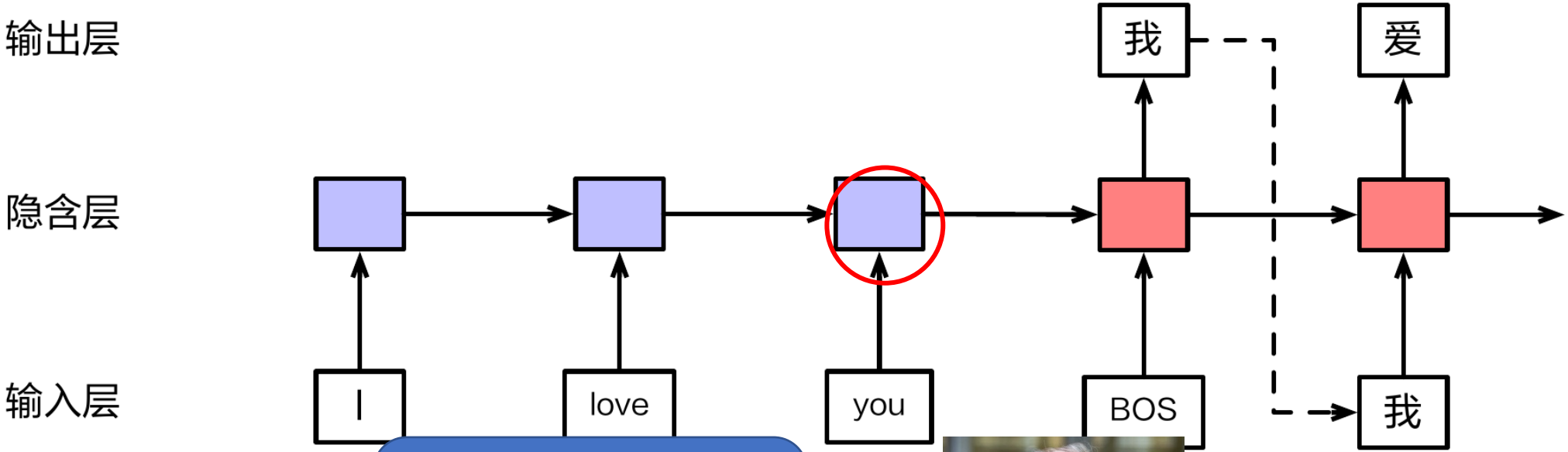
PyTorch实现：LSTM的两种输出

```
lstm = nn.LSTM(8, 16, batch_first=True)
x = torch.randn(2, 6, 8)
out, (hn, cn) = lstm(x)
print(out.shape)  # [2, 6, 16]
print(hn.shape)   # [1, 2, 16]
print(cn.shape)   # [1, 2, 16]
logits = nn.Linear(16, 2)(hn[-1])
```

用途	使用的状态
句子分类	最终有效状态 hn[-1]
词性标注	所有有效时刻的状态 out

变长序列使用 pack_padded_sequence，避免将Padding当作正文继续递推。

编码源句，再逐步生成目标句



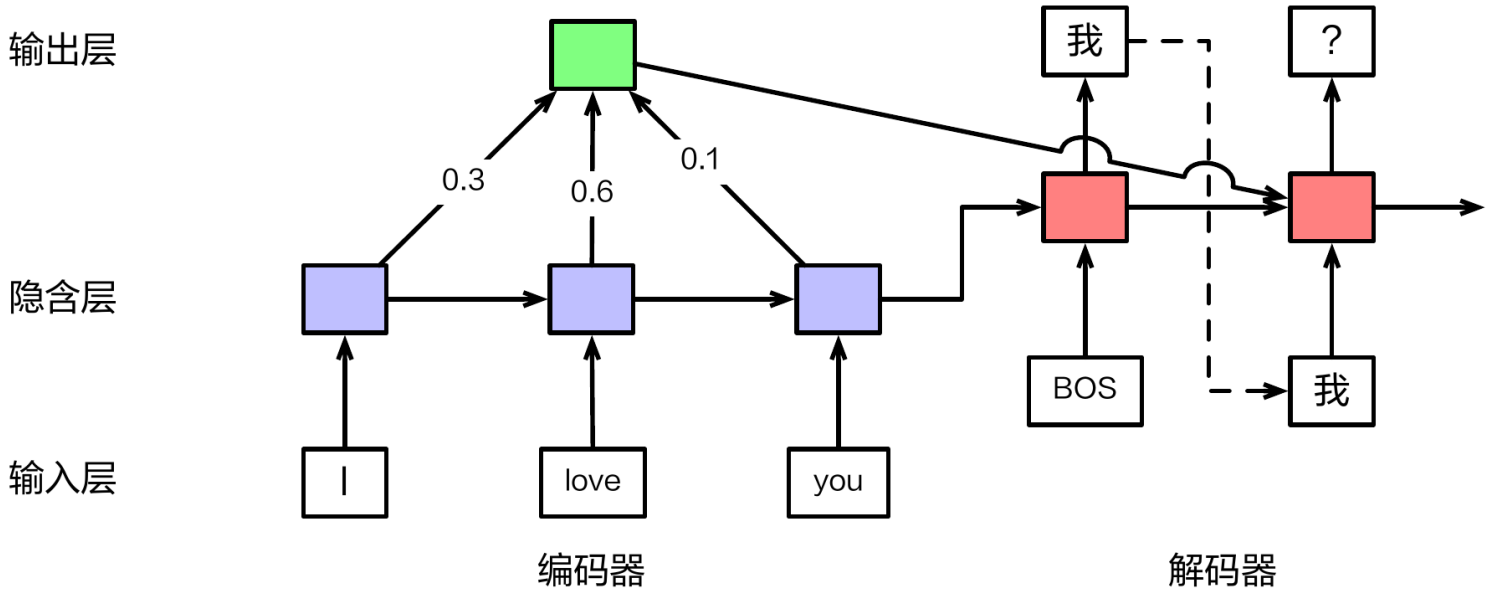
You can't cram the meaning of a whole %&!$ing sentence into a single $&!*ing vector!



Ray Mooney

长句的细节都经过一个固定维状态，可能难以保留。

注意力：生成时重新读取源句



打分

当前需求与源词匹配

归一化

Softmax得到权重

加权求和

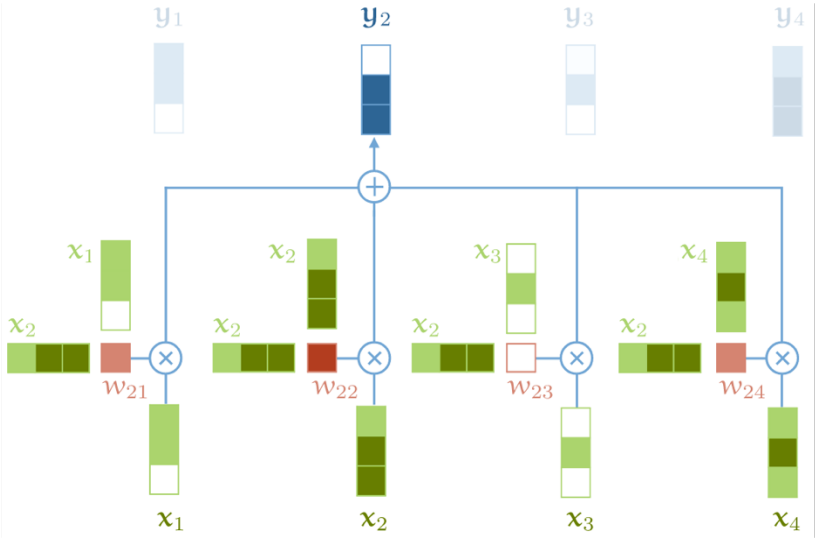
读出相关的源信息

$$c = 0.3h_1 + 0.6h_2 + 0.1h_3$$

图中权重为教学示意。每次生成的查询不同，读取的内容也可以不同。

自注意力模型

- 通过某一输入与周围输入的相关性（注意力）来更新该输入
  - “观其伴、知其义”
- 输入：n个向量构成的序列$\boldsymbol{x}_1, \boldsymbol{x}_2, \ldots, \boldsymbol{x}_n$
- 输出：每个向量对应的新向量$\boldsymbol{y}_1, \boldsymbol{y}_2, \ldots, \boldsymbol{y}_n$
- $\boldsymbol{y}_i = \sum_{j=1}^{n} \alpha_{ij} \boldsymbol{x}_j$，　α_{ij}为 $\boldsymbol{x}_i$ 与 $\boldsymbol{x}_j$之间的注意力值




```
>>> import torch
>>> import torch.nn.functional as F
>>> # 生成一个(2, 3, 4)的张量，第1维表示批次大小，第2维表示序列长度，第3维表示词
    向量维度
>>> x = torch.randn(2, 3, 4)
>>> # @是矩阵乘法运算符，保持参与运算张量前面的维度不变，最后两维进行矩阵乘法运算
>>> attn = x @ x.transpose(1, 2)
>>> attn = F.softmax(attn, dim=-1)
>>> y = attn @ x
>>> print(y, y.shape)
tensor([[[-0.0933,  0.1734, -0.4230, -0.2408],
         [-0.0098,  0.2139, -0.4638, -0.2008],
         [ 0.6867,  1.4132,  1.0766,  1.7611]],

        [[-0.1461,  0.0825,  0.7220, -1.6230],
         [ 1.4327, -0.3362,  0.3039, -0.6506],
         [-0.0056,  0.1133,  1.3207, -0.0676]]]) torch.Size([2, 3, 4])
```

Transformer

- Vaswani et al., **Attention** Is All You Need, NIPS 2017
- Transformer的翻译
  - 变压器?
  - 变形金刚?
- 自注意力模型还需要解决的几个问题
  - 没有考虑输入的位置信息
  - 输入向量 x_i 同时承担三种角色，不易学习
    - 计算注意力权重时的两个向量以及被加权的向量
  - 自注意力计算结果互斥，无法同时关注多个输入
  - 只考虑了两个输入向量之间的关系，无法建模多个向量之间的更复杂关系





融入位置信息

- 位置嵌入（Position Embeddings）
  - 类似词嵌入，每个绝对位置赋予一个连续、低维、稠密的向量表示
  - 向量参数参与模型学习
- 位置编码（Position Encodings）
  - 直接将一个整数（位置索引值）映射为一个向量

$$\mathrm{PosEnc}(p,i)=\begin{cases}\sin\left(\frac{p}{10000^{\frac{i}{d}}}\right) & \text{if } i \text{ is even}\\ \cos\left(\frac{p}{10000^{\frac{i-1}{d}}}\right) & \text{else}\end{cases}$$

- 输入向量 = 词向量 + 位置嵌入/编码

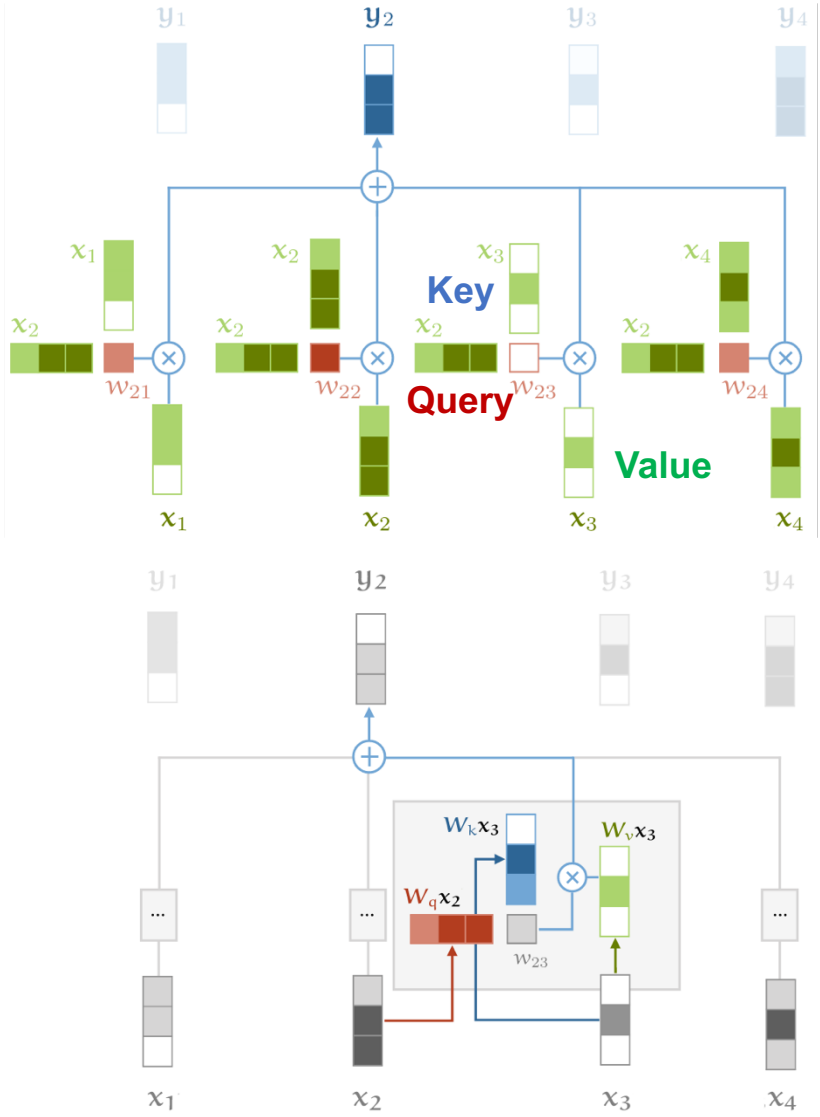
赋予输入向量角色信息

- 输入向量的三种角色
  - 查询（Query）
  - 键（Key）
  - 值（Value）
- 分别对输入向量进行线性映射
  - $\boldsymbol{q}_i = \boldsymbol{W}^{\mathrm{q}}\boldsymbol{x}_i$
  - $\boldsymbol{k}_i = \boldsymbol{W}^{\mathrm{k}}\boldsymbol{x}_i$
  - $\boldsymbol{v}_i = \boldsymbol{W}^{\mathrm{v}}\boldsymbol{x}_i$
- 新的自注意力模型

$$\boldsymbol{y}_i = \sum_{j=1}^{n} \alpha_{ij}\boldsymbol{v}_j$$

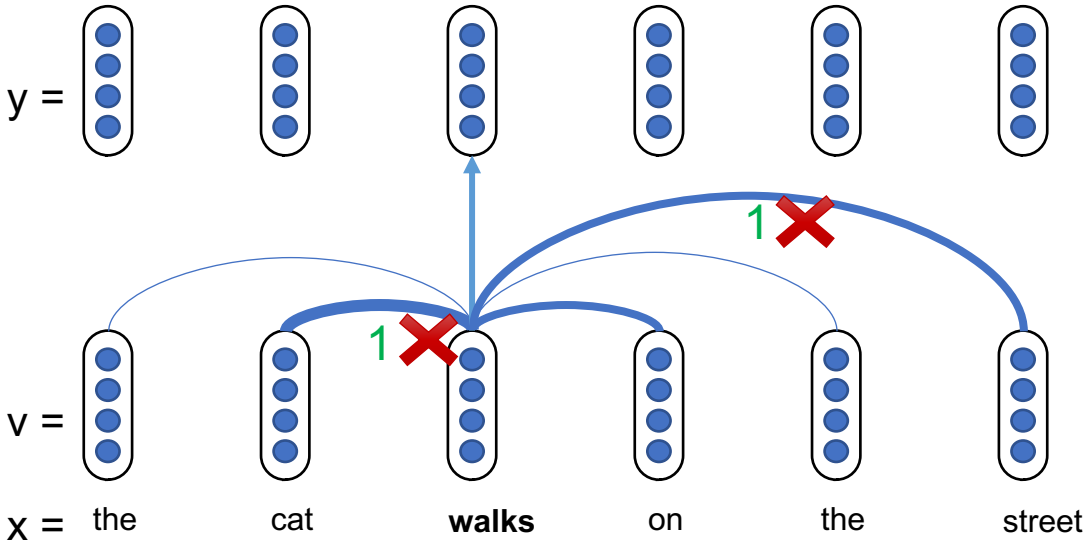
$$\alpha_{ij} = \mathrm{Softmax}(\hat{\boldsymbol{\alpha}}_i)_j$$

$$\hat{\alpha}_{ij} = \mathrm{attn}(\boldsymbol{q}_i, \boldsymbol{k}_j)$$

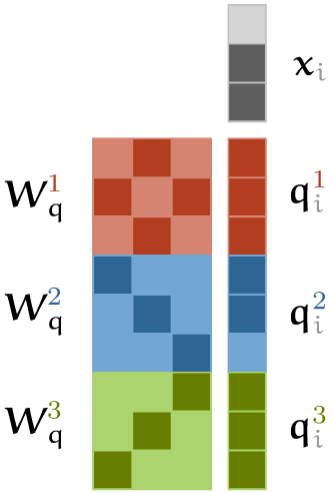


多头自注意力

- 由于Softmax函数的性质，无法使得多个自注意力分数趋近于1



- 使用多组自注意力模型产生多组不同的注意力结果
  - 设置多组输入映射矩阵
  - 类似使用多个卷积核提取不同的特征



多头自注意力的实现

```python
class MultiHeadSelfAttention(nn.Module):
    def __init__(self, config):
        super().__init__()
        self.config = config
        self.proj = nn.Linear(config.n_embd, config.n_embd * 3)

    def forward(self, x):
        B, T, C = x.size() # batch_size, seq_len, n_embd

        # 获得batch中每个输入的q, k, v, 并将q, k, v分解为n_head组
        q, k, v = self.proj(x).chunk(3, dim=-1)
        k = k.view(B, T, self.config.n_head, -1).transpose(1, 2)
        q = q.view(B, T, self.config.n_head, -1).transpose(1, 2)
        v = v.view(B, T, self.config.n_head, -1).transpose(1, 2)

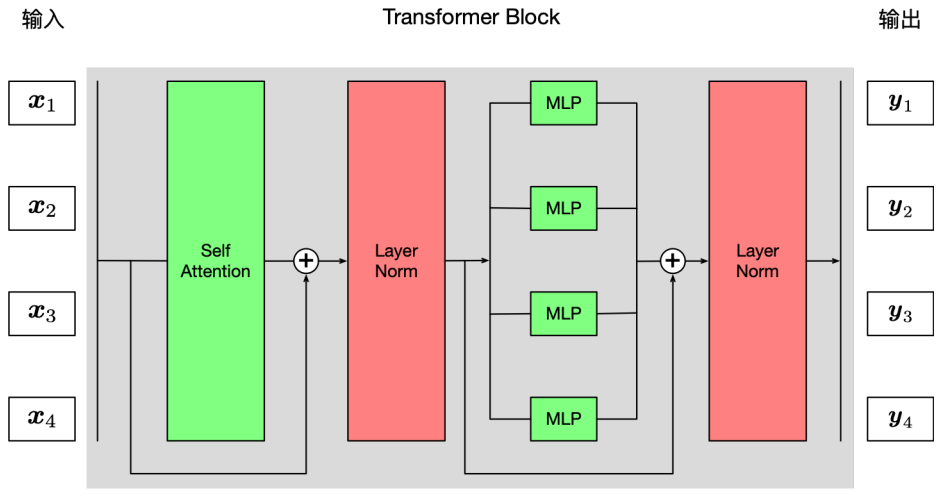
        # 计算自注意力:
        # (B, n_head, T, hs) x (B, n_head, hs, T) -> (B, n_head, T, T)
        attn = (q @ k.transpose(-2, -1)) / (k.size(-1) ** 0.5)
        attn = F.softmax(attn, dim=-1)
        y = attn @ v
        y = y.transpose(1, 2).reshape(B, T, C)
        return y
```

```python
@dataclass
class Config:
    batch_size: int = 2
    seq_len: int = 3
    n_embd: int = 4
    n_head: int = 2
```

```python
if __name__ == '__main__':
    config = Config()
    x = torch.randn(config.batch_size, config.seq_len, config.n_embd)
    self_attn = MultiHeadSelfAttention(config)
    y = self_attn(x)
    print(y, y.shape)
```

多层自注意力

- 原始自注意力模型仅考虑了任意两个向量之间的关系
- 如何建模高阶关系？
  - 直接建模高阶关系导致模型复杂度过高
  - 堆叠多层自注意力模型（消息传播机制）
- 增强模型的表示能力——增加非线性
  - 增加非线性的多层感知器模型（MLP）
- 使模型更容易学习
  - 层归一化（Layer Normalization）
  - 残差连接（Residual Connections）



四类模型处理文本的方式

模型	聚合信息的方式	优势	主要限制
平均池化MLP	词向量平均后分类	结构简单、计算快	忽略词序
CNN	局部窗口与池化	提取局部搭配	长距关系依赖感受野
RNN / LSTM	沿时间递推状态	显式处理顺序	串行计算、长程记忆有限
Transformer	位置间直接加权	全局交互、训练并行	全注意力开销随长度平方增长

同一任务的效果需要在相同数据划分、指标与计算预算下比较。

情感分类实战：数据如何进入模型

1 **划分样本** 固定训练、验证、测试集合

2 **建立词表** 只统计训练集，预留 <pad> 与 <unk>

3 **组织批次** Dataset取样本，collate_fn补齐并记录长度

4 **模型计算** Embedding、编码器、聚合、分类层

5 **比较结果** 同一指标与划分，检查错误样例

教材使用NLTK的sentence_polarity。中文短句只用于解释计算过程。

变长文本：Padding不应改变句子含义

样本	词元1	词元2	词元3	长度
句子甲	电影	很	精彩	3
句子乙	不好	看	<pad>	2
乙的有效位置	True	True	False	2

约定：<pad> = 0，<unk> = 1。

```
valid = ids.ne(0).unsqueeze(-1)
x = embedding(ids)
h = (x * valid).sum(1) / valid.sum(1)
```

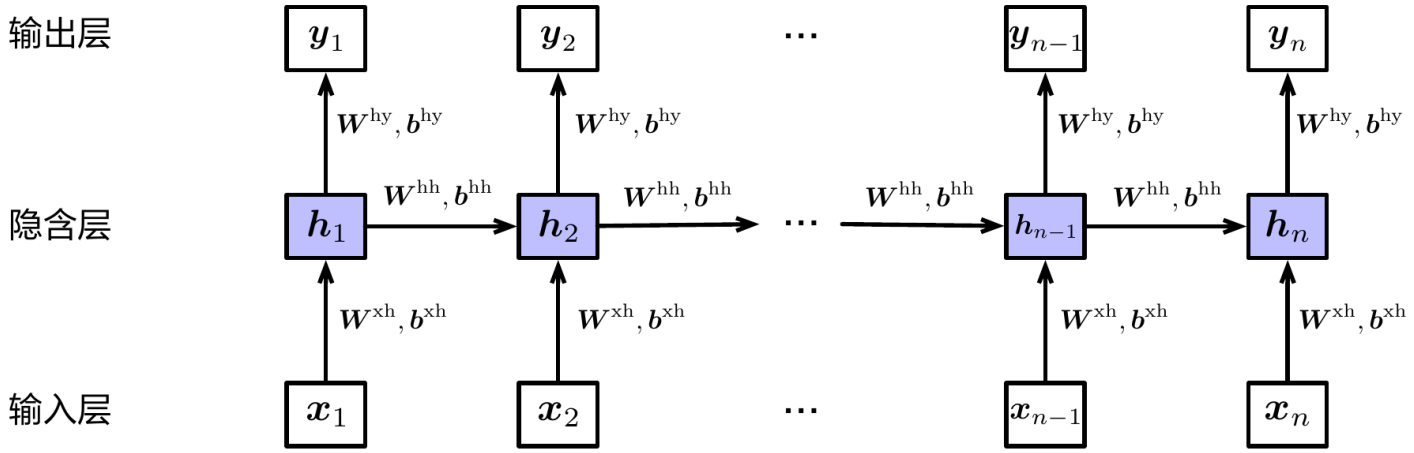
平均池化只除以有效词元数。此处假设每个样本至少含一个有效词元。

同一个分类任务，更换不同编码器

模型	编码器输出	句子表示	分类输出
平均池化MLP	词向量 [B,T,D]	有效词元平均 [B,D]	[B,2]
CNN	卷积响应 [B,F,L]	时间维最大值 [B,F]	[B,2]
单向LSTM	隐状态 [B,T,H]	最后有效状态 [B,H]	[B,2]
Transformer	上下文表示 [B,T,D]	有效位置平均 [B,D]	[B,2]

共享的训练目标：提高真实标签对应的得分。

词性标注：每个词元都有一个输出



词元	They	love	movies
词性	PRP	VBP	NNS

```
logits = tag_head(hidden)       # [B, T, C]
valid = ids.ne(0)               # [B, T]
loss = F.cross_entropy(logits[valid], tags[valid])
```

准确率也只统计有效词元。输入词表与词性标签表是两个不同的映射。

课后实验：比较四种情感分类模型

实验内容	记录什么
MLP、CNN、LSTM、Transformer	参数量、训练时间、验证与测试指标
相同划分、词表和标签映射	保证结果能够比较
不同随机种子重复实验	指标均值与波动
查看错误样例	否定句、转折句、长句分别有哪些错误

本章小结

模型结构

MLP组合特征，CNN提取局部模式
RNN递推状态，注意力直接汇聚上下文

训练过程

前向计算损失，反向传播求梯度
优化器更新参数，验证集帮助选择方案

任务迁移

句子分类输出一个标签
序列标注为每个有效词元输出标签

谢 谢!

理解语言，认知社会

以中文技术，助民族复兴



长按二维码，关注哈工大SCIR

微信号：HIT_SCIR